Programmcode und Änderungen sichern

PROACADEMY
Einfach erklärt. Zum Nachschlagen und Ausdrucken.

Git hält Änderungen am Projekt fest. GitHub speichert die hochgeladenen Versionen online. So lässt sich nachvollziehen, was geändert wurde und welche Version zur Veröffentlichung vorgesehen ist.

Projekt auf GitHub öffnen

Begriff	Einfach erklärt
Repository / Repo	Der Projektordner mit seiner Änderungshistorie.
Branch	Ein eigener Entwicklungsstand. Hier: start und main.
Commit	Eine lokal gespeicherte Gruppe von Änderungen.
Push	Überträgt lokale Commits zu GitHub.
Deployment	Veröffentlicht eine App-Version bei Cloudflare.

> **GitHub ist keine vollständige Datensicherung des Portals.** Supabase-Daten, private PDF-Dateien, laufende Stripe-Abos und geheime Zugangsdaten sind dadurch nicht gesichert.

Für Mitarbeiter ist GitHub vor allem eine Ablage für Handbücher und Änderungswünsche. Technische Änderungen und Veröffentlichungen übernimmt die zuständige Administration.

Handbücher und Versionen finden

1. Das verlinkte Repository öffnen. Falls nötig mit dem eigenen berechtigten GitHub-Konto anmelden.
2. Oben im Dateibereich den Branch auswählen. **start** wird für die Entwicklung genutzt, **main** für den übernommenen Stand. Vor einem Vergleich prüfen, welchen Branch du gerade siehst.
3. README.md öffnen. Von dort führen Links zu Handbüchern und Projektdokumentation. Die geplanten späteren Verbesserungen stehen in **V2.0.md**.
4. Für die Änderungshistorie die Commit-Ansicht öffnen. Datum, Beschreibung und geänderte Dateien helfen bei der Suche. Eine Commit-ID bezeichnet genau einen gespeicherten Stand.

> Ein neues Handbuch auf deinem Mac erscheint erst nach Commit und Push auf GitHub. Eine Datei auf GitHub beweist wiederum nicht, dass dieser Stand schon bei Cloudflare veröffentlicht wurde.

Was wohin gehört

In das Projekt	Getrennt und geschützt
Quellcode, Dokumentation, Tests	Zugangstokens und Passwörter
Handbuch-PDFs und Textfassungen	Kundenexporte und persönliche Berichte
V2.0-Wunschliste	Datenbank- und Storage-Sicherungen

Die aktuellen Plattform-Handbücher werden zusätzlich lokal in den gewünschten Portal-Ordner kopiert. Diese Kopie ist unabhängig von GitHub.

Nützliche Befehle zum Nachsehen

Technische Administration: Terminal im Projektordner öffnen. Diese Befehle lesen den lokalen Git-Stand und ändern keinen Programmcode.

```
git status --short
git branch --show-current
git log -5 --oneline
git diff
```

Befehl	Was du ablesen kannst
status --short	Welche Dateien noch geändert oder neu sind.
branch --show-current	Auf welchem Branch du arbeitest.
log -5 --oneline	Die letzten fünf gespeicherten Änderungen.
diff	Die noch nicht vorgemerkten Änderungen an bekannten Dateien.

Neue, noch nicht erfasste Dateien werden in git diff nicht mit ihrem Inhalt angezeigt. Sie stehen bei git status mit zwei Fragezeichen.

> Wenn Dateien mit Zusätzen wie „ 2.ts“ auftauchen, erst prüfen, ob es Kopien aus einer Ordnersynchronisierung sind. Nicht automatisch mit hochladen und nicht ungeprüft löschen.

Der Befehl git fetch origin lädt Informationen vom entfernten Repository, verändert aber keine Arbeitsdateien. Zusammenführen oder Branchwechsel sind eigene Schritte.

Projektgrundlage: aktueller Git-Workflow mit start/main und manuellen Cloudflare-Deployments.

Änderungen bewusst hochladen

Nur nach Auftrag für Commit und Push. Vorher Inhalt prüfen und den vorgesehenen Branch bestätigen. Dateien gezielt auswählen, damit keine privaten Exporte mitkommen.

Beispiel: ein vorhandenes Handbuch ändern

```
git branch --show-current
git status --short
git add docs/BENUTZERHANDBUCH.md
git add docs/ProAcademy-Benutzerhandbuch.pdf
git diff --cached --stat
git commit -m "Benutzerhandbuch aktualisieren"
git push origin start
```

Dieses Beispiel setzt voraus, dass der aktuelle Branch **start** ist und genau diese beiden Dateien zum Auftrag gehören. Bei anderen Änderungen werden die passenden Dateien ausdrücklich ausgewählt.

GitHub: Commits zum entfernten Repository übertragen

> Ein Push überträgt den Code. Der derzeitige Projektablauf veröffentlicht anschließend separat bei Cloudflare. Eine spätere automatische Build-Verbindung könnte das ändern und muss dann ausdrücklich dokumentiert werden.

- [ ] Richtigen Branch und ausgewählte Dateien geprüft.
- [ ] Keine .env-Dateien, Tokens oder Kundendaten im Commit.
- [ ] Push erfolgreich; Commit auf GitHub sichtbar.
- [ ] Übernahme nach main und Deployment nur im beauftragten Umfang.

Bei Konflikten oder abgelehntem Push die Administration einschalten. Kein force push und kein Zurücksetzen mit Datenverlust als einfache Fehlerlösung.